

CampusMesh
Internet-Independent Peer-to-Peer Campus Communication Network
Project Report
Programming in Java
Submitted by
Suryansh Pratap Singh
Registration No.: 25BAI10845
CSE (AI & ML) — 2nd Year

VIT Bhopal University

Abstract

Most of us take it for granted that a message will reach whoever we send it to. But on a large campus, that assumption breaks down quickly — a dead zone in the hostel, a router that goes down during exams, or simply too many devices fighting for the same Wi-Fi signal. CampusMesh started as an attempt to answer a simple question: if the internet disappeared for an hour, could nearby devices still talk to each other? The result is a Java prototype where devices form their own small network, passing messages along from one device to the next until they reach their destination. Along the way it picks up a few useful abilities — routing around broken links, holding onto a message until the recipient comes back online, sounding an alert to everyone nearby in an emergency, and checking that a transferred file hasn't been corrupted. None of it needs the internet.

1. Introduction

Campus connectivity is rarely as reliable as we'd like. Coverage gaps, network outages, and sheer congestion during peak hours all get in the way of something as basic as sending a message to someone a few buildings away. In an emergency, this stops being a minor annoyance and becomes a real problem. CampusMesh is a small step toward fixing that: instead of relying on a central network, every device taking part becomes a node that can pass messages on to its neighbours. If two devices can't reach each other directly, whichever devices sit in between simply relay the message forward, and the network keeps working even when no single link in it is very reliable.

2. Objectives

The project set out to do a handful of concrete things:

Build a working peer-to-peer communication system, not just a theoretical model.

Let devices discover each other and form a network of nodes.

Route messages across multiple hops when a direct link isn't available.

Hold onto messages for offline recipients and deliver them once they reconnect.

Give the network a way to broadcast urgent SOS alerts.

Move files between devices with a way to confirm they weren't corrupted in transit.

Keep a log of what's happening on the network for debugging and review.

Use the project as a hands-on way to apply core Java concepts — OOP, multithreading, exception handling — rather than just studying them.

3. Technologies Used

4. System Architecture

Rather than cram everything into one class, the system is split into layers that each do one job. A command-line interface takes in what the user types; an application layer figures out what that command actually means; a services layer does the real work — sending messages, moving files, syncing state, computing routes; and underneath all of that sits the peer-to-peer network itself, just a collection of nodes talking to their neighbours.

Command Line UI ■ Application Layer ■ Network / Services (Messaging, Files, Sync, Routing) ■
Peer-to-Peer Network (Node <-> Node <-> Node)

5. Core Modules

5.1 Multi-Hop Routing

This is really the heart of the project. When two devices can't talk directly, the router treats the network as a graph and runs a Breadth-First Search to find the shortest chain of hops between them — so a message from A can still reach D by passing through B and C, even though A and D have never been in range of each other.

5.2 Store-and-Forward Messaging

Devices go offline all the time — phones die, laptops sleep, people just walk out of range. Rather than lose the message, the system holds it in storage until the recipient reappears on the network, at which point it's delivered automatically. It's a small feature, but it's what makes the network feel dependable rather than fragile.

5.3 SOS / Emergency Alerts

For emergencies, waiting for a routed, one-to-one message isn't good enough — you want the alert to reach as many people as possible, as fast as possible. The SOS command does exactly that, pushing a message out to every node the network can currently reach.

5.4 File Transfer with Integrity Verification

Sending a file is easy; trusting that it arrived intact is the harder part. Before a file leaves the sender, it's run through SHA-256 to produce a hash. After it arrives, the same hash is computed again — if the two match, nothing was lost or corrupted along the way.

5.5 Multithreaded Networking

A server that can only talk to one device at a time isn't much of a network. Each incoming connection gets handed off to its own worker thread, so the server can keep several conversations going at once without one slow connection holding up the rest.

6. Implementation Snapshot

Once built with Maven, the whole thing runs as a command-line application. There's no GUI to speak of — everything happens through simple commands, which keeps the focus on the network logic rather than the interface:

```
mvn clean package mvn exec:java
```

A few of the commands you'd actually use day-to-day:

7. Testing Summary

The system was run through a set of scenarios covering the situations it's meant to handle — not just the happy path, but failures too:

8. Limitations and Future Work

It's worth being upfront about what this project isn't. Right now, the peer-to-peer behaviour is simulated over local TCP/IP — there's no actual Bluetooth or Wi-Fi Direct discovery happening between physical devices yet. That's the obvious next step, along with an Android app so it could realistically run on the phones people already carry. Beyond that, encryption, smarter (weighted) routing, and even long-range options like LoRa are all reasonable directions if this were taken further.

9. Conclusion

What this project shows, at least at prototype scale, is that you don't need the internet to build something genuinely useful for campus communication. Peer-to-peer networking, routing that adapts to broken links, messages that wait patiently for offline recipients, and file transfers you can actually trust — put together, they add up to a small but resilient system. Just as importantly, building it meant putting object-oriented design, concurrency, and graph algorithms to real use, rather than leaving them as textbook ideas.

Project Details

Technology	Purpose
Java	Core programming language
Maven	Build and dependency management
TCP/IP, ServerSocket	Local network communication
Multithreading	Concurrent connection handling
BFS	Multi-hop route discovery
SHA-256	File integrity verification
CLI	User interaction

Command	Function
addnode / connect	Build the network topology

Command	Function
route	Compute a path between two nodes
send / sos	Deliver a normal or emergency message
file	Transfer a file with checksum verification
sync / logs / status	Synchronize state and inspect system activity

Test Case	Result
Direct message delivery	Message reaches destination
Multi-hop message	Message follows computed route
Offline destination	Message stored and later delivered
File transfer	File transferred and checksum verified
SOS broadcast	Alert reaches all reachable nodes
Node/server failure	Handled gracefully without crash

Field	Detail
Name	Suryansh Pratap Singh
Registration No.	25BAI10845
Branch	CSE AI & ML
Year	2nd Year
Institution	VIT Bhopal University
Course	Programming in Java